

EECS 182 Cheatsheet: Attention & Diffusion Models

A narrative-style mathematical summary for last-minute review

1 Introduction

This cheatsheet summarizes the essence of two of the most important pillars of modern deep learning studied in EECS 182:

- **Attention and Transformer architectures**, which define the computational backbone of modern language and vision models.
- **Diffusion models**, which currently dominate generative AI through iterative denoising and score estimation.

The goal is to flow from intuition to equations, and then to practical insights, forming a coherent conceptual narrative.

2 Scaled Dot-Product Attention

We begin with the fundamental mechanism underlying Transformers. Given queries Q , keys K , and values V , attention computes a weighted combination of values according to similarity between queries and keys:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

The division by $\sqrt{d_k}$ stabilizes variance of logits. If $q, k \in \mathbb{R}^{d_k}$ have Gaussian entries, then

$$\mathbb{E}[q^\top k] = 0, \quad \text{Var}(q^\top k) = d_k.$$

Thus, without scaling, logits grow with dimension and saturate the softmax. The scaling ensures logits remain $\mathcal{O}(1)$.

Softmax—unlike argmax—maintains differentiability, enabling gradients to flow through the attention mechanism. This becomes essential when learning queries and keys.

3 Self-Attention and Cross-Attention

A Transformer layer relies on **self-attention**, where all tokens attend to each other within the same sequence:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

This mechanism enables modeling long-range dependencies independent of spatial or temporal locality.

In encoder–decoder architectures, **cross-attention** allows decoder representations to attend to encoder outputs:

$$Q = YW_Q, \quad K = HW_K, \quad V = HW_V.$$

This is critical for tasks requiring conditioning on another modality or sequence.

For autoregressive generation, **causal masking** ensures token i cannot attend to future token $j > i$.

4 Multi-Head Attention

Realizing that different subspaces of representation space may encode different kinds of structure, Transformers split attention into h heads:

$$\text{head}^{(m)} = \text{Attention}(Q^{(m)}, K^{(m)}, V^{(m)}),$$

with projections $W_Q^{(m)}, W_K^{(m)}, W_V^{(m)}$. The outputs concatenate and mix via

$$\text{MHA}(X) = \text{Concat}(\text{head}^{(1)}, \dots, \text{head}^{(h)})W_O.$$

Multi-head attention thus learns multiple relational perspectives simultaneously. Its computational bottleneck lies in forming the $n \times n$ attention matrix, scaling as $O(n^2)$.

5 The Transformer Block

A single encoder block forms a micro-network:

$$\begin{aligned} X' &= \text{MHA}(X), & \tilde{X} &= \text{LayerNorm}(X + X'), \\ Z &= \text{MLP}(\tilde{X}), & Y &= \text{LayerNorm}(\tilde{X} + Z). \end{aligned}$$

Residual connections preserve gradient flow. Layer normalization stabilizes activations across training. The structure elegantly alternates between relational reasoning (attention) and local feature transformation (MLP).

Decoder blocks additionally include masked self-attention and cross-attention.

6 Efficient Attention Variants

Large-scale sequence models motivate more efficient designs:

- **Multi-Query Attention (MQA):** all heads share keys and values. Reduces memory traffic while preserving performance.
- **Kernelized/linear attention:** rewrite softmax as a kernel

$$\exp(q^\top k) = \phi(q)^\top \phi(k),$$

approximate ϕ with random features, and use associativity to reduce complexity from $O(n^2)$ to $O(n)$.

These variants replace expensive pairwise interactions with more scalable approximations.

7 Generative Models in Context

Generative models aim to capture $p_\theta(x)$:

- **Likelihood-based** (e.g., VAEs, flows): optimize $\log p_\theta(x)$ directly.
- **Implicit** (e.g., GANs): generate samples without tractable likelihood.
- **Diffusion/score-based models:** learn denoising dynamics or scores to iteratively refine noise into data.

Diffusion models unify ideas from energy-based modeling, variational inference, and stochastic differential equations.

8 Variational Inference and the ELBO

Given a latent variable model with prior $p(z)$ and likelihood $p_\theta(x|z)$, we have:

$$\log p_\theta(x) = \log \int p_\theta(x|z)p(z) dz.$$

Introducing an approximate posterior $q_\phi(z|x)$ and applying Jensen’s inequality yields the **Evidence Lower Bound (ELBO)**:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \parallel p(z)).$$

Diffusion models will reuse this idea, but in a time-indexed form across noise levels.

9 Diffusion Models: Forward Process

A diffusion model defines a Markov chain that gradually corrupts data with Gaussian noise:

$$q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}x_{t-1}, \beta_t I).$$

Strikingly, one can sample directly from $q(x_t|x_0)$:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

where $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$.

This reparameterization enables efficient training across all timesteps.

10 Diffusion Models: Reverse Process

Generation requires learning the reverse-time dynamics:

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_\theta(t)).$$

The original DDPM formulation fixes Σ_θ and parameterizes μ_θ using a **noise-predicting network**:

$$\epsilon_\theta(x_t, t) \approx \epsilon.$$

Learning to predict noise effectively learns how to undo the diffusion process.

11 Training Objective: The Simple Loss

The ELBO for diffusion decomposes into KL terms at each timestep. This simplifies to a remarkably elegant training loss:

$$L_{\text{simple}} = \mathbb{E}_{x_0, t, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2].$$

Thus, the network learns to denoise by predicting the exact noise added in the forward step.

12 Score Matching View

Diffusion can also be viewed as learning **scores**, the gradients of log-densities:

$$s_\theta(x, t) \approx \nabla_x \log p_t(x).$$

For Gaussian forward noise, the score of $q(x_t|x_0)$ is analytically known, enabling **denoising score matching**:

$$L_{\text{DSM}} = \mathbb{E} [\|s_\theta(x_t, t) - \nabla_{x_t} \log q(x_t|x_0)\|^2].$$

The noise-prediction network and score model differ only by a known scaling transformation. This dual interpretation connects diffusion to energy-based modeling.

13 Sampling: DDPM and DDIM

A standard reverse sampler performs:

$$x_{t-1} = \mu_{\theta}(x_t, t) + \sigma_t z, \quad z \sim \mathcal{N}(0, I),$$

yielding a slow but faithful generative process.

Deterministic variants such as **DDIM** skip stochasticity while preserving marginals, enabling much faster sampling via large steps across the diffusion schedule.

14 Guidance

Conditional generation modifies scores to emphasize desired features.

Classifier Guidance

$$\tilde{s}(x_t, t, y) = s_{\theta}(x_t, t) + w \nabla_x \log p_{\phi}(y|x_t).$$

Classifier-Free Guidance

Train model both with and without condition y :

$$\tilde{\epsilon}_{\theta} = (1 + w)\epsilon_{\theta}(x_t, t, y) - w\epsilon_{\theta}(x_t, t, \emptyset).$$

This approach dominates text-to-image generation.

15 Practical Considerations

Key engineering details include:

- **Variance schedule** β_t : linear or cosine schedules control difficulty of denoising.
- **SNR balancing** improves training stability across timesteps.
- **Parameterization choices** (noise vs. x_0 vs. v prediction).
- **Compute cost**: large UNets, long diffusion chains, and large datasets require significant compute.

16 Rapid Review Checklist

- Attention formula: $\text{softmax}(QK^{\top}/\sqrt{d_k})V$.
- Why scaling? Variance control for stable training.
- Transformer block: $\text{MHA} \rightarrow \text{Add+LN} \rightarrow \text{MLP} \rightarrow \text{Add+LN}$.
- Diffusion forward: $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$.
- Noise-prediction loss: $\|\epsilon - \epsilon_{\theta}(x_t, t)\|^2$.
- Score viewpoint: gradients of log densities.